SMART TASK MANAGER

A SUMMER INTERNSHIP REPORT

Submitted by

Abinaya U – 810023205017



DEPARTMENT OF INFORMATION TECHNOLOGY

ANNA UNIVERSITY,

TIRUCHIRAPPALLI – 620 024.

UNIVERSITY COLLEGE OF ENGINEERING (BIT CAMPUS),

ANNA UNIVERSITY, TIRUCHIRAPPALLI – 620 024

DEPARTMENT OF INFORMATION TECHNOLOGY



BONAFIDE CERTIFICATE

This is to certify that this Summer Internship project report **"Smart Task Manager"** is the bonafide work of **Abinaya U (810023205017)** who has satisfactorily completed the course of **IT3711 – SUMMER INTERNSHIP** for **SEVENTH** semester, **B.TECH INFORMATION TECHNOLOGY** during the academic year **2026-2027**.

Faculty InchargeHead of the Department

Submitted for the practical examination held on _____

ABSTRACT

The Smart Task Manager is a full stack web application that helps users plan, prioritise and track their daily work. Individuals and small teams often keep tasks in notes, chat messages and spreadsheets, which leads to missed deadlines and unclear priorities. This project brings task creation, categorisation, deadlines, reminders and progress tracking into one secure web application.

The application is built with the MERN stack: React for the front end, Node.js and Express for the back end, and MongoDB for storage. Authentication uses JSON Web Tokens (JWT) with hashed passwords. A priority-scoring algorithm suggests which task to do next based on its importance and due date, and a dashboard summarises completed, pending and overdue work. The interface is responsive and works on desktop and mobile browsers.

The main objectives are to build a complete full stack application, develop REST APIs with CRUD operations, secure user data with authentication, provide smart priority suggestions and deadline alerts, and present task statistics through a dashboard. By replacing scattered lists with one organised system, the project improves productivity, reduces missed deadlines and supports better time management.

Overall, the project demonstrates the practical use of the complete web development life cycle – requirement analysis, UI design, API development, database design, authentication, testing and deployment – using tools widely used in industry.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
1	**INTRODUCTION** 1.1 Objectives of the Internship, 1.2 Motivation, 1.3 Scope, 1.4 Organisation	3
2	**COMPANY PROFILE**	5
3	**DAILY WORK REPORT**	7
4	**BLOCK DIAGRAM**	9
5	**SMART TASK MANAGER SYSTEM** 5.1 Existing System, 5.2 Proposed System, 5.3 Capabilities, 5.4 System Requirements	9
6	**PROPOSED METHOD** 6.1 Requirement Analysis to 6.8 Database Design	12
7	**ALGORITHM** 7.1 User Authentication to 7.7 Insight Generation	14
8	**CODE IMPLEMENTATION** 8.1 Project Setup to 8.10 Deployment	16
9	**SAMPLE OUTPUT** 9.1 Registration and Login to 9.11 Dashboard	20
10	**ANALYSIS** 10.1 Functional Analysis to 10.6 Challenges and Solutions	24
11	**CONCLUSION** Learning Outcomes	26
12	**FUTURE WORK**	27
13	**REFERENCES**	28

1. INTRODUCTION

Smart Task Manager is a full stack web application for managing personal and team tasks. Many people still depend on paper lists, chat messages or spreadsheets, which have no reminders, no priority guidance and no progress view. As a result, deadlines are missed and effort is spent on the wrong tasks.

The primary purpose of this project is to build a secure, responsive and easy-to-use application where users can register, log in, create tasks with deadlines and priorities, organise them by category, and monitor progress through a dashboard. The "smart" feature ranks pending tasks using a priority score so that users immediately see what needs attention first.

The project is developed using React for the user interface, Node.js with Express for the REST API, MongoDB with Mongoose for the database, JWT for authentication, and Git and GitHub for version control. The system follows a three-tier architecture consisting of the Presentation Tier (React), the Application Tier (Express API) and the Data Tier (MongoDB). Key modules include User Authentication, Task Management, Smart Priority Suggestions, Reminders and Overdue Detection, and Dashboard Reporting.

The project gives practical knowledge of the web development life cycle, REST API design, database modelling, authentication and responsive design. It can be extended with notifications, team collaboration and machine learning based suggestions in future versions.

1.1 OBJECTIVES OF THE INTERNSHIP

The main objective is to design and develop a complete full stack web application and to apply modern web technologies to a real problem. The objectives of the project are:

- To design and develop a full stack web application from requirement analysis to deployment.
- To build a responsive user interface with reusable React components.
- To develop REST APIs for create, read, update and delete (CRUD) operations using Node.js and Express.
- To store and retrieve data using MongoDB and Mongoose.
- To implement secure registration and login using JWT and password hashing.
- To implement smart priority suggestions, reminders and overdue detection.
- To create a dashboard that shows task statistics and progress.
- To test the application with Postman and manual test cases and deploy it online.
- To demonstrate the practical application of JavaScript, React, Node.js, MongoDB and Git.

1.2 MOTIVATION

The motivation behind this project is to make everyday task planning simple and intelligent. Many existing tools are either too complex or paid, while simple lists give no help in deciding what to do first. A lightweight smart task manager helps users stay organised, meet deadlines and reduce stress.

Another motivation is to gain hands-on experience with the complete web stack. The project provided practical exposure to front-end design, back-end API development, database design and security, using tools widely used in industry.

1.3 SCOPE OF THE PROJECT

The scope of this project covers a multi-user web application with authentication, task CRUD, categories, priorities, due dates, search and filters, a priority-based Smart Suggestions list and a statistics dashboard. Real-time collaboration, native mobile applications and AI-based prediction are outside the scope of this version.

1.4 ORGANISATION OF THE REPORT

The report is organised into thirteen chapters. Chapter 1 introduces the project and its objectives. Chapter 2 describes the company. Chapter 3 gives the daily work report. Chapters 4 to 7 explain the block diagram, the existing and proposed systems, the method and the algorithms. Chapter 8 presents the code, and Chapter 9 shows the sample outputs. Chapter 10 gives the analysis and test results, Chapter 11 the conclusion, Chapter 12 the future work and Chapter 13 the references.

2. COMPANY PROFILE



About the Company

Innoknowvex Pvt Ltd is a technology and training company based in Bengaluru, Karnataka. It provides software, data and technology-based solutions and offers industrial training and internship programmes that connect academic learning with real industry practice.

During the internship, students work on real-time projects under the guidance of an industry expert. Exposure includes requirement analysis, development, testing,

deployment and documentation. Internships follow a structured project-based approach with regular review by the team leader.

As part of my internship at Innoknowvex Pvt Ltd, I worked on Smart Task Manager under the guidance of Mr. D. Kumaresan, Team Leader. The project involved designing the application, building the React front end, developing REST APIs with Node.js and Express, designing the MongoDB database, implementing JWT authentication, testing the application and deploying it. The internship improved my skills in JavaScript, React, Node.js, MongoDB and problem solving, and helped me understand how full stack applications are built in an industry environment.

Company Details

Item	Details
Company Name	Innoknowvex Pvt Ltd
Address	Sb01-no 581/a, Rupena Agrahara, Sector IV, HSR Layout, Bengaluru, Karnataka – 560102, India
Industry	Information Technology, Training and Software Solutions
Industry Expert (Joint Supervisor)	Mr. D. Kumaresan, Team Leader
Internship Domain	Full Stack Web Development
Project Completed	Smart Task Manager
Internship Period	12 June 2026 – 12 July 2026

Technologies Used

Area	Tools
Programming Languages	JavaScript (ES6+), HTML5, CSS3
Front End	React, React Router, Axios, Tailwind CSS
Back End	Node.js, Express.js
Database	MongoDB, Mongoose
Security	JWT, bcrypt
Development Tools	VS Code, Postman, MongoDB Compass
Version Control	Git and GitHub

3. DAILY WORK REPORT

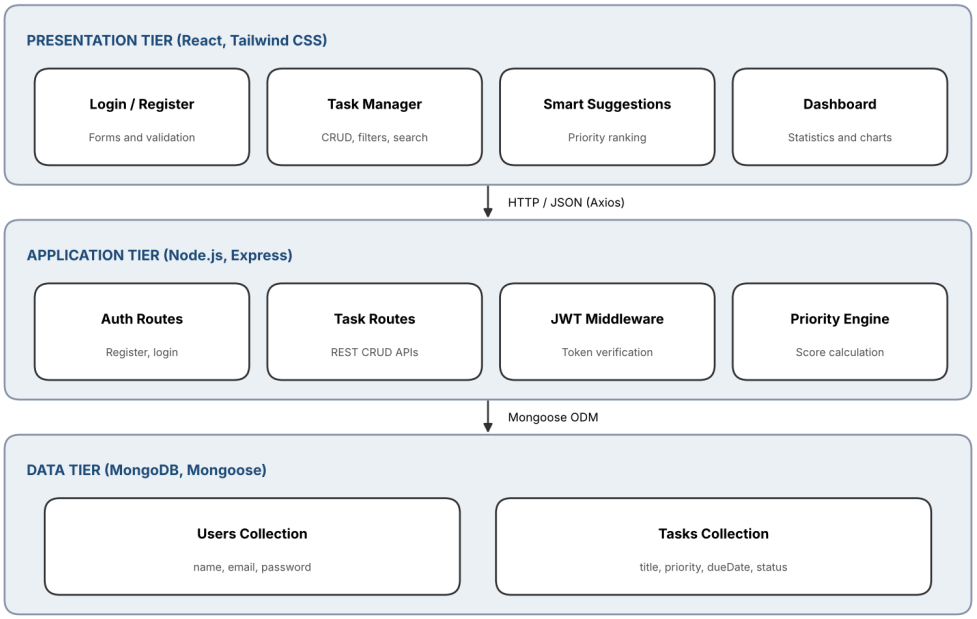
Day	Work Done
Day 1	Attended orientation; briefed on internship guidelines, evaluation criteria and reporting format. Discussed and finalised the project topic "Smart Task Manager" with the industry expert.
Day 2	Gathered requirements by discussing with the mentor. Listed the users (student, professional, team member) and the key features the application must provide.

Day	Work Done
Day 3	Reviewed existing task management tools. Documented the drawbacks of paper lists and spreadsheets.
Day 4	Finalised the scope and objectives of the project. Prepared the list of hardware and software requirements.
Day 5	Set up the environment: Node.js, VS Code, MongoDB, Postman and Git. Verified the setup with a sample Express server.
Day 6	Revised HTML, CSS and modern JavaScript (ES6): arrow functions, promises, async/await and modules.
Day 7	Designed the overall architecture using Presentation, Application and Data tiers. Prepared the block diagram.
Day 8	Created wireframes for the login, task list and dashboard pages.
Day 9	Designed the database schema with User and Task collections and prepared the ER diagram.
Day 10	Initialised the Express server, organised the folder structure and connected to MongoDB.
Day 11	Created the Mongoose models for User and Task with validation rules.
Day 12	Implemented user registration with bcrypt password hashing.
Day 13	Implemented user login and JWT token generation.
Day 14	Created authentication middleware to protect the task routes.
Day 15	Built the Create and Read task APIs and tested them with Postman.
Day 16	Built the Update and Delete task APIs; added input validation and error handling.
Day 17	Added search, filter and sort options to the task API.
Day 18	Set up the React project with React Router and the component folder structure.
Day 19	Built the login and register pages with form validation.
Day 20	Built the task list, task card and add/edit task form components.
Day 21	Connected React to the API using Axios and managed login state with Context API.
Day 22	Implemented the priority score algorithm and the Smart Suggestions list.
Day 23	Added due-date reminders and overdue highlighting.
Day 24	Built the statistics API and the dashboard with KPI cards and charts.
Day 25	Applied Tailwind CSS and made the interface responsive for mobile screens.
Day 26	Added loading states, error messages and success notifications.
Day 27	Performed API testing with Postman and manual testing of all pages; fixed bugs.
Day 28	Deployed the back end and front end to a hosting platform and verified the live application.
Day 29	Prepared the key findings and recommendations; took screenshots for documentation.
Day 30	Validated results, fixed issues and compiled the final internship report.

Weekly Summary

Week	Days	Work Completed
Week 1	Day 1 – 7	Orientation, requirement gathering, scope, environment setup, JavaScript revision, architecture design
Week 2	Day 8 – 14	Wireframes, database schema and ER diagram, Express setup, models, registration, login and JWT middleware
Week 3	Day 15 – 21	Task CRUD APIs, search and filters, React setup, login pages, task components, API integration
Week 4	Day 22 – 28	Priority algorithm, reminders, dashboard, responsive design, notifications, testing and deployment
Final days	Day 29 – 30	Key findings and recommendations, validation, documentation and final report

4. BLOCK DIAGRAM



Block diagram of the Smart Task Manager system

The React client sends requests to the Express API. The JWT middleware verifies the user, the controllers apply the business logic, and Mongoose reads or writes data in MongoDB. The response is returned as JSON and displayed by React as task lists, suggestions and dashboard charts.

5. SMART TASK MANAGER SYSTEM

5.1 Existing System

Most people manage tasks through paper notes, chat messages, calendars and basic spreadsheets. These methods are quick to start but offer little help in planning. This existing approach suffers from several limitations:

- **Scattered tasks** – Tasks are spread across notes, chats and files, which makes tracking difficult.
- **No priority guidance** – Lists show tasks but do not indicate which one should be done first.
- **No reminders** – Deadlines are missed because nothing alerts the user.
- **No progress view** – Users cannot see how many tasks are completed, pending or overdue.
- **Weak security** – Shared files and chats are not user-specific or protected.
- **Complex alternatives** – Full project tools are heavy or paid for simple personal needs.
- **No categorisation** – Work, study and personal tasks are mixed together.
- **Poor accessibility** – Notes are not available on all devices in an organised form.

These drawbacks show the need for a centralised, secure and intelligent task management system.

5.2 Proposed System

The proposed system is a web application with secure accounts, a central task database, category and priority management, a smart ranking of pending tasks, deadline alerts and a statistics dashboard. It runs in any browser and adapts to mobile screens. Key aspects of the proposed system include:

- **Centralised data** – All tasks are stored per user in a MongoDB database.
- **Secure access** – Passwords are hashed and every request is verified with a JWT.
- **Task management** – Tasks can be created, edited, completed, deleted, searched and filtered.
- **Smart suggestions** – A priority score ranks pending tasks by importance and urgency.
- **Reminders** – Due-soon and overdue tasks are highlighted automatically.
- **Dashboard** – Totals and completion rate give a quick view of progress.

By combining these features, the system reduces missed deadlines and helps users manage their time better.

5.3 Capabilities

- **User Management** – Registers users and logs them in securely.
- **Task Management** – Creates, updates, completes and deletes tasks.
- **Categorisation** – Groups tasks by category such as Study, Work and Personal.
- **Priority Management** – Assigns Low, Medium or High priority to each task.
- **Smart Suggestions** – Recommends the most important tasks to do next.
- **Search and Filter** – Finds tasks by keyword, status, category and priority.
- **Reminders** – Highlights due-soon and overdue tasks.
- **Dashboard** – Shows total, completed, pending and overdue tasks with completion rate.
- **Responsive Design** – Works on desktop, tablet and mobile screens.

5.4 System Requirements

The project was developed and tested on a standard personal computer. The hardware and software requirements are given below.

Hardware Requirement

Item	Specification
Processor	Intel Core i3 or higher
RAM	8 GB or more
Storage	256 GB SSD or higher
Display	1366 x 768 resolution or higher

Software Requirement

Item	Specification
Operating System	Windows 10 / 11
Programming Language	JavaScript (Node.js 18 or later)
Front End	React, Tailwind CSS
Back End	Express.js
Database	MongoDB 6.0
Development Environment	VS Code, Postman, MongoDB Compass
Version Control	Git and GitHub
Browser	Google Chrome

5.5 Advantages of the Proposed System

- Keeps all tasks in one secure and organised place.
- Helps users focus on the most urgent and important work.

- Reduces missed deadlines through highlighting and reminders.
- Gives a clear view of progress through the dashboard.
- Can be used from any device with a browser.
- Uses free and widely available tools, so the cost of development is low.

5.6 Limitations

- The priority score is rule-based and does not learn from user behaviour.
- Reminders appear inside the application; email or push alerts are not included.
- Team collaboration and task sharing are limited in this version.
- The application needs an internet connection to access stored tasks.

6. PROPOSED METHOD

6.1 Requirement Analysis

- Identifies the users, features and constraints of the application.
- Prepares use cases for register, login, manage tasks, view suggestions and view dashboard.
- Defines the data to be stored for users and tasks.

6.2 UI Design

- Prepares wireframes for the login, task list, task form and dashboard pages.
- Uses reusable React components for forms, task cards, filters and charts.
- Uses Tailwind CSS to build a clean and responsive layout.

6.3 Back-End Development

- Creates the Express server with modular routes, controllers and middleware.
- Develops REST endpoints: POST /api/auth/register, POST /api/auth/login, GET and POST /api/tasks, PUT and DELETE /api/tasks/:id, GET /api/tasks/stats.
- Validates input and returns clear error messages.

6.4 Authentication and Security

- Hashes passwords with bcrypt before storing them.
- Issues a JWT at login and verifies it in middleware before any task route.
- Filters every database query by the authenticated user so that data stays private.

6.5 Front-End Integration

- Uses React Router for page navigation and Axios for API calls.
- Stores the logged-in user and task state using the Context API.
- Shows loading states, error messages and success notifications.

6.6 Smart Features

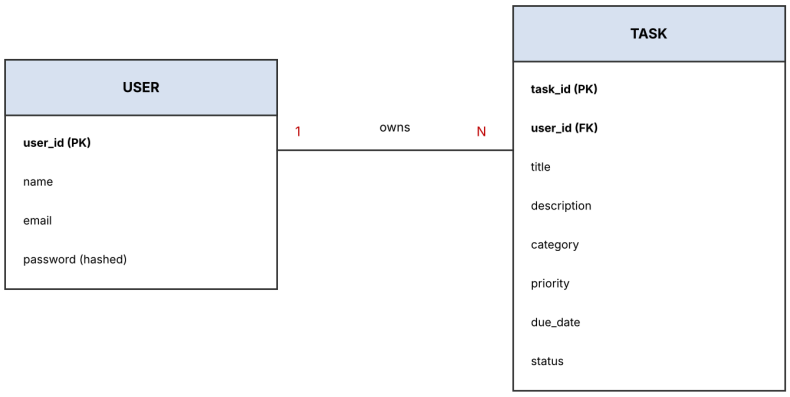
- Calculates a priority score for each pending task from its priority and due date.
- Sorts tasks by score to produce the Smart Suggestions list.
- Marks tasks as due soon or overdue based on the current date.

6.7 Testing and Deployment

- Tests every API with Postman and every page with manual test cases.
- Fixes bugs and checks the layout on different screen sizes.
- Deploys the back end and front end to a hosting platform.

6.8 Database Design

The data is stored in MongoDB using two collections. The User collection stores account details and the Task collection stores the tasks. Each task holds a reference to the user who owns it, so one user can have many tasks (1 : N). This keeps every user's data separate and consistent.



ER Diagram of the Smart Task Manager Database

Data Dictionary

Field	Data Type	Description
user_id	ObjectId	Unique identifier of the user
name	String	Full name of the user
email	String	Unique email address used to log in
password	String	Password stored as a bcrypt hash
task_id	ObjectId	Unique identifier of the task
title	String	Short title of the task
description	String	Optional details of the task
category	String	Category such as Study, Work or Personal
priority	String	Low, Medium or High
due_date	Date	Date by which the task must be completed

Field	Data Type	Description
status	String	Pending or Completed
created_at	Date	Derived: automatic timestamp when the task is created

7. ALGORITHM

7.1 User Registration – Algorithm

1. Receive name, email and password from the registration form.
2. Validate that all fields are present and the email format is correct.
3. Check that the email is not already registered.
4. Hash the password using bcrypt.
5. Save the user in the database and return a success message.

7.2 User Login – Algorithm

1. Receive email and password.
2. Find the user by email.
3. Compare the entered password with the stored hash.
4. If they match, create a JWT containing the user id.
5. Return the token to the client; otherwise return an error.

7.3 Task Management – Algorithm

1. Receive the request with the JWT in the Authorization header.
2. Verify the token and extract the user id.
3. Validate the task fields.
4. Create, read, update or delete the task belonging to that user only.
5. Return the result as JSON.

7.4 Priority Score – Algorithm

1. Fetch all pending tasks of the user.
2. Assign a weight to priority: High = 3, Medium = 2, Low = 1.
3. Compute days left = due date − today.
4. Assign urgency: overdue = 5, within 1 day = 4, within 3 days = 3, within 7 days = 2, otherwise 1.
5. Compute score = (priority weight × 2) + urgency.
6. Sort tasks by score in descending order and show the top tasks.

7.5 Reminder and Overdue Detection – Algorithm

1. Read the due date of each pending task.

2. Compare it with the current date.
3. Mark the task as Overdue if the due date has passed.
4. Mark the task as Due Soon if it is due within 2 days.
5. Display the matching highlight on the task card.

7.6 Dashboard Statistics – Algorithm

1. Count the total number of tasks of the user.
2. Count completed, pending and overdue tasks.
3. Compute completion rate = completed ÷ total × 100.
4. Return the figures and display them as KPI cards and charts.

7.7 Insight Generation – Algorithm

1. Review the dashboard figures and the suggestion list.
2. Identify overdue tasks and categories with the most pending work.
3. Link each pattern to a recommended action.
4. Present the recommendations to the user.

8. CODE IMPLEMENTATION

8.1 Database Schema (schema.sql)

Creates the two tables with the constraints described in Chapter 6

```
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    title TEXT NOT NULL,
    description TEXT DEFAULT '',
    category TEXT DEFAULT 'General',
    priority TEXT NOT NULL CHECK (priority IN ('Low', 'Medium', 'High')),
    due_date TEXT NOT NULL,
    est_hours REAL DEFAULT 1,
    status TEXT NOT NULL DEFAULT 'Pending'
        CHECK (status IN ('Pending', 'In Progress', 'Completed')),
    created_at TEXT NOT NULL,
    completed_at TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

8.2 Core Module – Database and Task Functions (core.py)

Opens the database, creates the tables and implements add, list, status update and delete operations.

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true }
});

const taskSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
  title: { type: String, required: true, trim: true },
  description: String,
  category: { type: String, default: "General" },
  priority: { type: String, enum: ["Low", "Medium", "High"], default: "Medium" },
  dueDate: Date,
  status: { type: String, enum: ["Pending", "Completed"], default: "Pending" }
}, { timestamps: true });
```

8.3 Core Module – Smart Engine (core.py)

Implements the urgency points, smart score, ranking, daily plan, reminders and statistics

```
router.post("/register", async (req, res) => {
  const { name, email, password } = req.body;
  if (await User.findOne({ email }))
    return res.status(400).json({ message: "Email already registered" });
  const hash = await bcrypt.hash(password, 10);
  await User.create({ name, email, password: hash });
  res.status(201).json({ message: "Registered successfully" });
});

router.post("/login", async (req, res) => {
  const user = await User.findOne({ email: req.body.email });
  if (!user || !(await bcrypt.compare(req.body.password, user.password)))
    return res.status(400).json({ message: "Invalid credentials" });
  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: "1d" });
  res.json({ token, name: user.name });
});
```

8.4 Flask Application – Authentication (app.py)

Creates the application, the login_required decorator and the registration, login and logout routes

```
from datetime import date
from functools import wraps
from flask import Flask, render_template, request, redirect, url_for, session, flash
from werkzeug.security import generate_password_hash, check_password_hash
import core

CATEGORIES = ["Academic", "Internship", "Personal", "Health", "General"]


def create_app(db_path=":memory:", today=None):
    app = Flask(__name__)
    app.config.update(SECRET_KEY="change-me", TODAY=today)
    app.db = core.connect(db_path)

    def now():
        return app.config["TODAY"] or date.today()
```

```
def login_required(view):
    @wraps(view)
    def wrapper(*a, **kw):
        if "uid" not in session:
            return redirect(url_for("login"))
        return view(*a, **kw)
    return wrapper

@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        f = request.form
        try:
            app.db.execute(
                "INSERT INTO users (name, email, password_hash) VALUES (?,?,?)",
                (f["name"], f["email"].lower(),
                 generate_password_hash(f["password"])))
            app.db.commit()
        except Exception:
            flash("Email is already registered.")
            return render_template("login.html", mode="register")
        return redirect(url_for("login"))
    return render_template("login.html", mode="register")

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        row = app.db.execute("SELECT * FROM users WHERE email = ?",
                             (request.form["email"].lower(),)).fetchone()
        if row and check_password_hash(row["password_hash"],
request.form["password"]):
            session["uid"], session["name"] = row["id"], row["name"]
            return redirect(url_for("dashboard"))
        flash("Invalid email or password.")
    return render_template("login.html", mode="login")

@app.route("/logout")
def logout():
    session.clear()
    return redirect(url_for("login"))
```

8.5 Flask Application – Task Routes (app.py)

Implements the dashboard, task list, add task, status and delete routes.

```
@app.route("/")
@login_required
def dashboard():
    tasks = core.get_tasks(app.db, session["uid"])
    plan, used = core.daily_plan(tasks, now())
    return render_template("dashboard.html", s=core.stats(tasks, now()),
                           rem=core.reminders(tasks, now()),
                           top=core.ranked(tasks, now())[:5],
                           plan=plan, used=used, today=now())

@app.route("/tasks")
@login_required
```

```
    def tasks():
        status = request.args.get("status") or None
        category = request.args.get("category") or None
        rows = core.get_tasks(app.db, session["uid"], status, category)
        for t in rows:
            t["score"] = core.smart_score(t, now())
            t["days_left"] = core.days_left(t, now())
        return render_template("tasks.html", tasks=rows, cats=CATEGORIES,
                               status=status, category=category)

    @app.route("/tasks/add", methods=["GET", "POST"])
    @login_required
    def add():
        if request.method == "POST":
            f = request.form
            core.add_task(app.db, session["uid"], f["title"], f["priority"],
                          f["due_date"], f.get("est_hours", 1),
                          f.get("category", "General"),
                          f.get("description", ""), now())
            flash("Task added.")
            return redirect(url_for("tasks"))
        return render_template("form.html", cats=CATEGORIES, today=now())

    @app.route("/tasks/<int:tid>/status", methods=["POST"])
    @login_required
    def status(tid):
        core.set_status(app.db, tid, session["uid"], request.form["status"], now())
        return redirect(request.referrer or url_for("tasks"))

    @app.route("/tasks/<int:tid>/delete", methods=["POST"])
    @login_required
    def delete(tid):
        core.delete_task(app.db, tid, session["uid"])
        return redirect(url_for("tasks"))

    return app


if __name__ == "__main__":
    create_app("tasks.db").run(debug=True)
```

8.6 Dashboard Template (dashboard.html)

Shows the statistic cards, reminders, top-priority tasks and the suggested daily plan.

```
{% extends "base.html" %}{% block body %}
<h2 style="margin-top:0">Welcome, {{ session.name }}
  <span class="small">({{ today.strftime("%d %b %Y") }})</span></h2>
<div class="cards">
  <div class="card"><h3>{{ s.total }}</h3><span>Total tasks</span></div>
  <div class="card"><h3>{{ s.completed }}</h3><span>Completed</span></div>
  <div class="card"><h3>{{ s.in_progress + s.pending }}</h3><span>Open
   tasks</span></div>
  <div class="card"><h3 style="color:#dc2626">{{ s.overdue
   }}</h3><span>Overdue</span></div>
  <div class="card"><h3>{{ s.completion_rate }}%</h3><span>Completion rate</span>
    <div class="bar"><i style="width:{{ s.completion_rate }}%"></i></div></div>
```

```
</div>
<div class="panel"><h2>Reminders</h2>
{% for t, msg in rem %}
  <div class="alert {{ '' if 'OVERDUE' in msg else 'warn' }}">
    <b>{{ t.title }}</b> &ndash; {{ msg }}</div>
{% else %}<p>No urgent tasks.</p>{% endfor %}</div>
<div class="row">
  <div class="panel"><h2>Top Priority (Smart Score)</h2>
    <table><tr><th>Task</th><th>Priority</th><th>Due</th><th>Score</th></tr>
    {% for t in top %}
    <tr><td>{{ t.title }}</td>
        <td><span class="tag {{ t.priority }}">{{ t.priority }}</span></td>
        <td>{{ t.due_date }}</td><td><b>{{ t.score }}</b></td></tr>
    {% endfor %}</table></div>
  <div class="panel"><h2>Suggested Plan for Today ({{ used }} h of 6 h)</h2>
    <table><tr><th>#</th><th>Task</th><th>Hours</th></tr>
    {% for t in plan %}
    <tr><td>{{ loop.index }}</td><td>{{ t.title }}</td><td>{{ t.est_hours }}</td></tr>
    {% endfor %}</table></div>
</div>
{% endblock %}
```

8.7 Add Task Template (form.html)

Shows the form used to enter a new task

```
{% extends "base.html" %}{% block body %}
<div class="panel" style="max-width:620px;margin:auto"><h2>Add New Task</h2>
<form method="post">
  <label>Title</label><input name="title" required>
  <label>Description</label><textarea name="description" rows="2"></textarea>
  <div class="row">
    <div><label>Category</label>
      <select name="category">{% for c in cats %}<option>{{ c }}</option>{% endfor
   %}</select></div>
    <div><label>Priority</label>
      <select name="priority"><option>High</option><option selected>Medium</option>
        <option>Low</option></select></div>
    <div><label>Due date</label>
      <input type="date" name="due_date" value="{{ today }}" required></div>
    <div><label>Estimated hours</label>
      <input type="number" step="0.5" min="0.5" name="est_hours" value="1"></div>
  </div>
  <button>Save Task</button>
</form></div>
{% endblock %}
```

8.8 Automated Tests – Core Logic (test_stm.py)

Eleven tests check the task functions, scoring, ranking, planner, reminders, statistics and database constraints

```
import unittest
from datetime import date
import core
from app import create_app
```

```
TODAY = date(2026, 7, 6)


class CoreTests(unittest.TestCase):
    def setUp(self):
        self.conn = core.connect()
        self.conn.execute(
            "INSERT INTO users (name,email,password_hash) VALUES ('A','a@x.com','h')")
        self.conn.commit()

    def add(self, title, pr, due, h=1):
        return core.add_task(self.conn, 1, title, pr, due, h, today=TODAY)

    def test_01_add_and_list(self):
        self.add("Write report", "High", "2026-07-08")
        self.assertEqual(len(core.get_tasks(self.conn, 1)), 1)

    def test_02_status_update(self):
        tid = self.add("Task", "Low", "2026-07-20")
        core.set_status(self.conn, tid, 1, "Completed", TODAY)
        self.assertEqual(core.get_tasks(self.conn, 1)[0]["completed_at"], "2026-07-06")

    def test_03_delete(self):
        tid = self.add("Task", "Low", "2026-07-20")
        core.delete_task(self.conn, tid, 1)
        self.assertEqual(core.get_tasks(self.conn, 1), [])

    def test_04_urgency_points(self):
        self.assertEqual([core.urgency_points(d) for d in (-3, 0, 2, 5, 20)],
                         [50, 40, 30, 20, 10])

    def test_05_score_priority_and_quick_win(self):
        t = {"priority": "High", "due_date": "2026-07-06", "est_hours": 1}
        self.assertEqual(core.smart_score(t, TODAY), 30 + 40 + 5)

    def test_06_ranking_order(self):
        self.add("Low far", "Low", "2026-07-30", 3)
        self.add("High today", "High", "2026-07-06", 2)
        self.add("Medium soon", "Medium", "2026-07-08", 2)
        names = [t["title"] for t in core.ranked(core.get_tasks(self.conn, 1), TODAY)]
        self.assertEqual(names, ["High today", "Medium soon", "Low far"])

    def test_07_completed_excluded_from_ranking(self):
        tid = self.add("Done", "High", "2026-07-06")
```

8.9 Automated Tests – Web Application (test_stm.py)

Six tests use the Flask test client to check login, access control, task creation, user isolation and password storage.

```
class WebTests(unittest.TestCase):
    def setUp(self):
        self.app = create_app(today=TODAY)
        self.c = self.app.test_client()

    def register_login(self, email="s@x.com"):
        self.c.post("/register", data={"name": "Siva", "email": email, "password":
   "pw123"})
        return self.c.post("/login", data={"email": email, "password": "pw123"})

    def test_12_login_required(self):
        self.assertEqual(self.c.get("/tasks").status_code, 302)

    def test_13_register_and_login(self):
        self.assertEqual(self.register_login().status_code, 302)
        self.assertEqual(self.c.get("/").status_code, 200)

    def test_14_wrong_password(self):
        self.c.post("/register",
                    data={"name": "S", "email": "s@x.com", "password": "pw123"})
        r = self.c.post("/login", data={"email": "s@x.com", "password": "bad"})
        self.assertIn(b"Invalid email or password", r.data)

    def test_15_add_task_via_form(self):
        self.register_login()
        self.c.post("/tasks/add", data={"title": "Demo", "priority": "High",
                                        "due_date": "2026-07-07", "est_hours": "2",
                                        "category": "Academic"})
        self.assertIn(b"Demo", self.c.get("/tasks").data)

    def test_16_users_are_isolated(self):
        self.register_login("a@x.com")
        self.c.post("/tasks/add", data={"title": "Secret", "priority": "Low",
                                        "due_date": "2026-07-30"})
        self.c.get("/logout")
```

```
        self.register_login("b@x.com")
        self.assertNotIn(b"Secret", self.c.get("/tasks").data)

    def test_17_password_not_plain(self):
        self.register_login()
        h = self.app.db.execute("SELECT password_hash FROM users").fetchone()[0]
        self.assertNotIn("pw123", h)

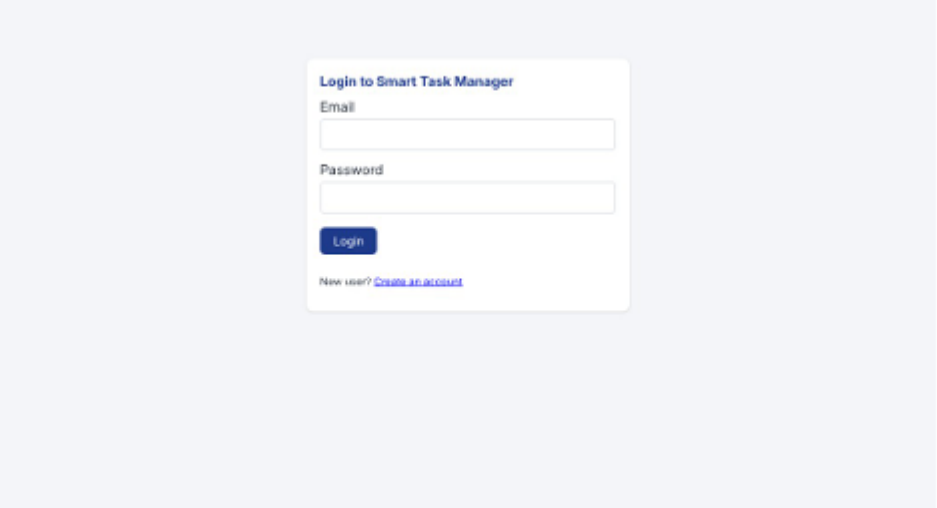

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

9. SAMPLE OUTPUT

The outputs below were produced by running the code in Chapter 8 on the Smart Task Manager application.

9.1 Registration and Login Output

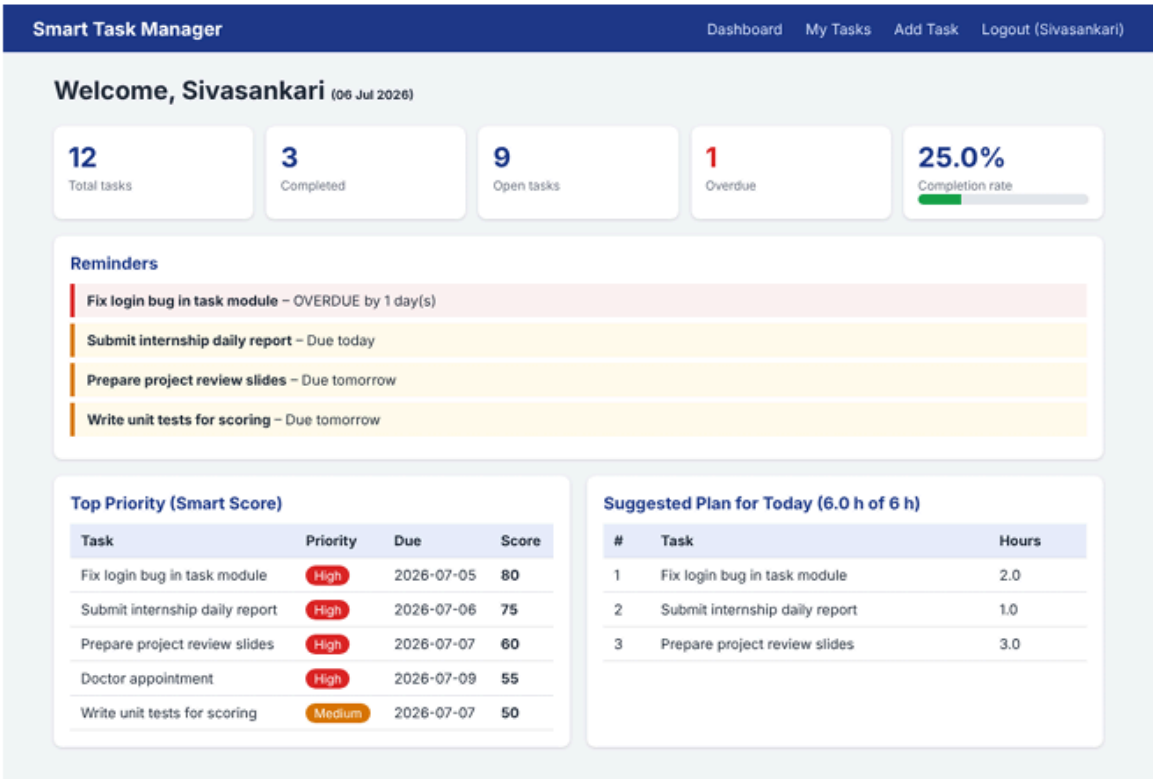
Displays the registration and login pages of the application.



Registration and Login Pages

9.2 Dashboard

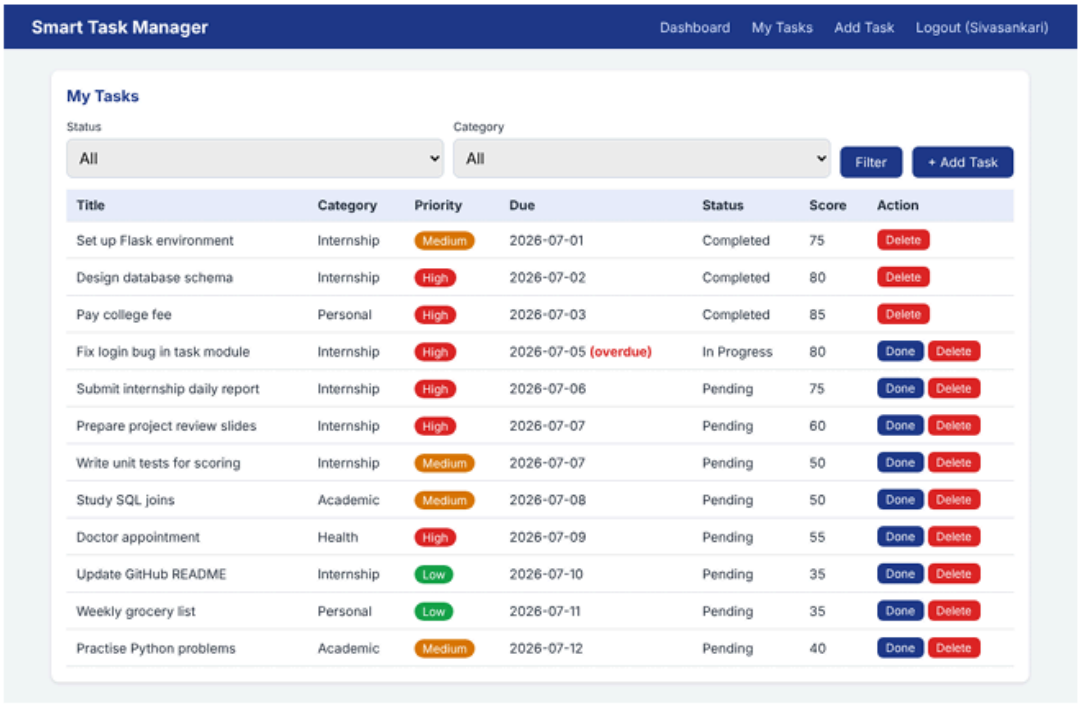
The dashboard shows task statistics, reminders, the top-priority tasks with their smart score and the suggested plan for the day.



Dashboard with reminders, ranking and daily plan

9.3 Task List

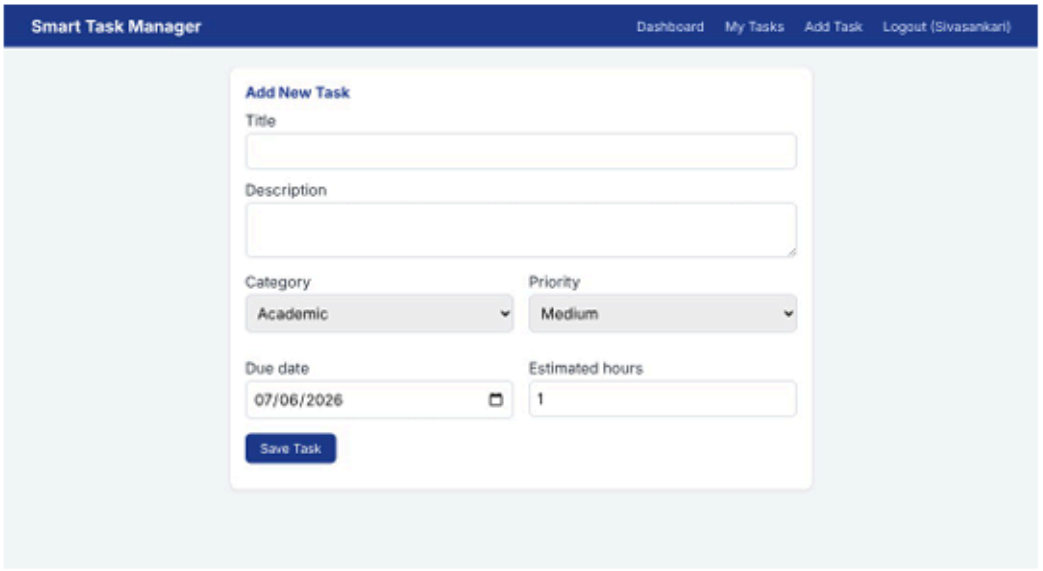
The task list shows every task with its priority, due date, status and smart score. The filters narrow the list by status or category.



Task List with filters and actions

9.4 Add and Edit Task

Shows the form used to create and edit a task.



Add Task Form

9.5 Smart Ranking Output

Open tasks ranked by smart score. The overdue task is first, followed by the High-priority tasks due soon.

```
$ python demo.py --rank

Task                            Priority Due          Days Score
Fix login bug in task module    High     2026-07-05    -1    80
Submit internship daily report  High     2026-07-06     0    75
Prepare project review slides   High     2026-07-07     1    60
Doctor appointment              High     2026-07-09     3    55
Write unit tests for scoring    Medium   2026-07-07     1    50
Study SQL joins                 Medium   2026-07-08     2    50
Practise Python problems        Medium   2026-07-12     6    40
Update GitHub README            Low      2026-07-10     4    35
Weekly grocery list             Low      2026-07-11     5    35
```

Ranked tasks with days left and smart score

9.6 Daily Plan Output

```
$ python demo.py --plan

Daily plan (capacity 6.0 h)
1. Fix login bug in task module        2.0 h
2. Submit internship daily report      1.0 h
3. Prepare project review slides       3.0 h
Total planned: 6.0 h
```

Suggested daily plan within 6 hours

9.7 Reminders and Statistics

```
$ python demo.py --reminders

Reminders for 2026-07-06
[OVERDUE by 1 day(s)] Fix login bug in task module
[Due today] Submit internship daily report
[Due tomorrow] Prepare project review slides
[Due tomorrow] Write unit tests for scoring
```

Reminder output

```
$ python demo.py --stats

total            12
completed        3
pending          8
in_progress      1
overdue          1
completion_rate  25.0
```

Task statistics

9.8 Test Results

All 17 automated tests pass.

```
$ python test_stm.py

test_01_add_and_list (CoreTests.test_01_add_and_list) ... ok
test_02_status_update (CoreTests.test_02_status_update) ... ok
test_03_delete (CoreTests.test_03_delete) ... ok
test_04_urgency_points (CoreTests.test_04_urgency_points) ... ok
test_05_score_priority_and_quick_win (CoreTests.test_05_score_priority_and_quick_win) ... ok
test_06_ranking_order (CoreTests.test_06_ranking_order) ... ok
test_07_completed_excluded_from_ranking (CoreTests.test_07_completed_excluded_from_ranking) ... ok
test_08_daily_plan_capacity (CoreTests.test_08_daily_plan_capacity) ... ok
test_09_reminders (CoreTests.test_09_reminders) ... ok
test_10_stats (CoreTests.test_10_stats) ... ok
test_11_invalid_priority_rejected (CoreTests.test_11_invalid_priority_rejected) ... ok
test_12_login_required (WebTests.test_12_login_required) ... ok
test_13_register_and_login (WebTests.test_13_register_and_login) ... ok
test_14_wrong_password (WebTests.test_14_wrong_password) ... ok
test_15_add_task_via_form (WebTests.test_15_add_task_via_form) ... ok
test_16_users_are_isolated (WebTests.test_16_users_are_isolated) ... ok
test_17_password_not_plain (WebTests.test_17_password_not_plain) ... ok

----------------------------------------------------------------------
Ran 17 tests in 1.127s

OK
```

Unit test run

10. ANALYSIS

The analysis was carried out stage by stage to check both the correctness of the features and the quality of the application.

10.1 Functional Analysis

Each module of the application – authentication, task management, smart suggestions, reminders and the dashboard – was tested with valid and invalid inputs. The REST APIs were tested in Postman and the pages were tested manually in Google Chrome. Every query is filtered by the authenticated user id, so a user can access only his or her own tasks.

The statistics returned by the API were compared with the task list shown on screen, and the totals matched, which confirmed the correctness of the dashboard calculations.

10.2 Test Cases

Each module was tested with normal and edge-case inputs. The results are given below.

No.	Module	Test Input	Expected Result	Result
1	Registration	Valid name, email and password	User is created	Pass
2	Registration	Email already registered	Error message is shown	Pass
3	Login	Correct email and password	JWT is issued and dashboard opens	Pass
4	Login	Wrong password	Login is rejected	Pass
5	Protected route	Request without token	401 Unauthorised response	Pass
6	Create task	Valid task data	Task is saved and listed	Pass
7	Update task	Change status to Completed	Status is updated	Pass
8	Delete task	Existing task id	Task is removed	Pass
9	Priority score	High priority task due tomorrow	Task appears at top of suggestions	Pass
10	Dashboard	Mix of pending and completed tasks	Counts match the task list	Pass
11	Responsive layout	Mobile screen width	Layout adjusts without breaking	Pass

10.3 Key Findings

- The application provides complete task management with secure, user-specific access.
- The priority score places overdue and high-priority tasks at the top of the suggestions.
- Passwords are stored only as bcrypt hashes and all task routes require a valid JWT.
- The dashboard gives an immediate view of total, completed, pending and overdue tasks.
- The responsive design keeps the interface usable on desktop and mobile screens.

10.4 Recommendations

- Add email or push notifications so that reminders reach users outside the application.
- Add team sharing and task assignment for group work.
- Use refresh tokens and rate limiting to strengthen security.
- Use the dashboard regularly to review pending and overdue work.

10.5 Comparison with the Manual Approach

Manual lists need constant rewriting, give no reminders and do not show which task is most important. The Smart Task Manager stores tasks in one place, ranks them automatically, highlights deadlines and shows progress visually, so that users can act on their work quickly.

Overall, the analysis confirms that the project meets its objectives of building a secure full stack application that supports better planning and time management.

10.6 Challenges Faced and Solutions

Challenge	Solution
CORS errors between React and Express	Enabled the cors middleware on the Express server
Losing login state after page refresh	Stored the token in the browser and restored the user in Context on load
Tasks visible to other users	Filtered every query by the authenticated user id
Date and time-zone differences	Stored dates in UTC and formatted them on the client
Layout breaking on small screens	Used responsive utility classes and flexible grids
Handling errors from the API	Added try/catch blocks and clear error messages in the interface

11. CONCLUSION

The Smart Task Manager project shows how a full stack web application can solve an everyday problem. By replacing scattered lists with one secure application and an intelligent priority ranking, the project reduces missed deadlines, saves effort and gives users a clear view of their work.

The project covered the full web development life cycle: requirement analysis, UI design, database design, REST API development, authentication, front-end integration, testing and deployment. Tools such as React, Node.js, Express, MongoDB and Git were used together.

The internship at Innoknowvex Pvt Ltd gave valuable practical experience in JavaScript, React, Node.js, MongoDB and full stack development. The work can serve as a foundation for more advanced applications, including real-time collaboration and intelligent recommendations.

Learning Outcomes

- Understood the complete web development life cycle from requirements to deployment.
- Gained hands-on skills in building responsive interfaces with React and Tailwind CSS.
- Learned to develop REST APIs using Node.js and Express.
- Learned to design and query MongoDB databases using Mongoose.
- Understood authentication, password hashing and protecting routes with JWT.
- Learned to test APIs with Postman and to use Git and GitHub for version control.
- Improved problem solving, documentation and teamwork skills in an industry environment.

12. FUTURE WORK

- Adding email, SMS and push notifications for reminders.
- Adding team workspaces, task assignment and comments.
- Supporting recurring tasks and calendar integration.
- Adding a drag-and-drop Kanban board.
- Using machine learning to learn user habits and improve suggestions.
- Building a Progressive Web App and mobile applications.
- Adding real-time updates using WebSockets.
- Strengthening security with refresh tokens, rate limiting and role-based access.

- Deploying with a CI/CD pipeline on a cloud platform such as AWS, Azure or GCP.
- Adding productivity reports sent automatically by email.

13. REFERENCES

1. React Documentation – Components, hooks and routing.
2. Node.js Documentation – Runtime and core modules.
3. Express.js Documentation – Routing and middleware.
4. MongoDB Documentation – Collections, queries and indexing.
5. Mongoose Documentation – Schemas and models.
6. JSON Web Token Introduction – Token-based authentication.
7. MDN Web Docs – HTML, CSS and JavaScript.
8. Tailwind CSS Documentation – Utility-first styling.
9. Postman Documentation – API testing.
10. GeeksforGeeks – Web development concepts.
11. W3Schools – JavaScript and web learning resources.